

ForexGuard

Real-Time Trader Anomaly Detection Engine

GitHub Repository: <https://github.com/Bhargava22-cpu/Forex-Guard>

1. Overview

ForexGuard is a real-time anomaly detection system for a forex brokerage environment. It monitors user behavior across the client portal and trading terminal, detects suspicious patterns using machine learning, and generates human-readable risk alerts for a compliance team.

The system is designed as a streaming pipeline — each incoming event is processed individually against the user's recent history, and an anomaly score with explanation is returned instantly via a REST API.

2. System Architecture

Layer	Description
Data generation	~50,000 synthetic events across 500 users spread across January 2026. Each user has an assigned active hour window. Normal events cluster within that window. Anomalies deliberately fall outside it.
Feature engineering	Raw events converted into behavioral signals — rolling z-scores, time deviation from personal baseline, IP deviation, PnL volatility, session metrics, instrument concentration
Modeling	Isolation Forest for point anomalies. LSTM Autoencoder for sequence anomalies. Both trained on engineered features.
Streaming pipeline	Events streamed via Kafka. Consumer forwards each event to the API. Features computed live against per-user history maintained in fakeredis.
API layer	FastAPI endpoint accepting event JSON, returning anomaly label, risk score, and human-readable explanation

Pipeline flow:

```
Producer -> Kafka -> Consumer -> POST /predict -> API
-> Update user history -> Compute features -> Isolation Forest
-> LSTM Autoencoder (if 10+ events) -> Risk score + reason -> Response
```

3. Repository Structure

```
forexguard/
|-- dataset.py           # Synthetic data generation
|-- feature_engineering.py # Batch + real-time feature logic
|-- models.py           # Isolation Forest + LSTM training
|-- reason.py           # Explainability - human-readable alerts
|-- api.py              # FastAPI prediction endpoint
|-- consumer.py         # Kafka consumer - forwards events to API
|-- producer.py         # Kafka producer - streams synthesised events
|-- forex_events.csv    # Generated raw dataset
|-- engineered_features.csv # Feature-engineered dataset
|-- final_predictions.csv # Model output on test set
|-- if_model.pkl        # Saved Isolation Forest
|-- lstm_model.keras    # Saved LSTM Autoencoder
|-- scaler.pkl          # Saved MinMaxScaler
```

```
`-- lstm_threshold.pkl          # Saved anomaly threshold
```

4. Setup Instructions

4.1 Install dependencies

```
pip install fastapi uvicorn pandas numpy scikit-learn tensorflow joblib fakeredis kafka
```

4.2 Generate dataset

```
python dataset.py
```

Produces `forex_events.csv` with ~50,000 events. Seed fixed at 42 for reproducibility.

4.3 Run feature engineering

```
python feature_engineering.py
```

Produces `engineered_features.csv` with all computed behavioral features.

4.4 Train models

```
python models.py
```

Trains both models, saves all artifacts, and prints classification reports.

4.5 Run the streaming demo

First time only — create Kafka container:

```
docker run -d --name kafka -p 9092:9092 \
  -e KAFKA_NODE_ID=1 \
  -e KAFKA_PROCESS_ROLES=broker,controller \
  -e KAFKA_CONTROLLER_QUORUM_VOTERS=1@localhost:9093 \
  -e KAFKA_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093 \
  -e KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 \
  -e KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT \
  -e KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER \
  -e KAFKA_INTER_BROKER_LISTENER_NAME=PLAINTEXT \
  -e KAFKA_AUTO_CREATE_TOPICS_ENABLE=true \
  -e CLUSTER_ID=Mku30EVBNTcwNTJENDM2Qg== \
  confluentinc/cp-kafka:7.6.0
```

Terminal 1 — Start Kafka:

```
docker start kafka
```

Terminal 2 — Start API:

```
uvicorn api:app --reload
```

Runs at <http://localhost:8000>. Interactive docs at <http://localhost:8000/docs>.

Terminal 3 — Start Consumer:

```
python consumer.py
```

Terminal 4 — Run Producer:

```
python producer.py
```

Watch Terminal 3 for live anomaly results.

5. Dataset

5.1 Overview

Fully synthetic dataset with ~50,000 events across 500 users, spread across January 2026. Each user starts at a random day within that month and events progress forward from there. Each user is assigned a personal active hour window at generation time. Normal events are constrained to that window so each user has a realistic and consistent behavioral baseline.

5.2 Event types

Event type	Source	Key fields
login	Client portal	ip_address, device, session_duration
deposit	Client portal	amount
withdrawal	Client portal	amount
trade	Trading terminal	trade_volume, lot_size, margin, instrument
kyc_update	Client portal	kyc_changed flag
account_modification	Client portal	account_modified flag

5.3 User behavior profiles

- Normal (70%) — 1 to 2 logins per day, small consistent trade volumes, stable IP, occasional deposits
- Active (20%) — frequent trades, multiple instruments, higher activity density
- Suspicious (10%) — irregular timing, large transactions, IP switching, KYC changes before withdrawals

5.4 Injected anomalies

Anomalies are injected at a 3% rate across five types:

- Amount anomaly — deposit or withdrawal 10 to 40x above normal (50,000 to 200,000)
- Trade anomaly — trade volume 10 to 20x the user's baseline with elevated margin
- Time anomaly — activity forced to a random hour more than 2 hours outside the user's personal active window. No hardcoded value — the off-hours time is derived from each user's own normal pattern
- IP anomaly — login from a 172.16.x.x address, distinct from the user's established base IP
- Behavior anomaly — KYC update immediately preceding a withdrawal, or IP switch on login

6. Feature Engineering

Feature engineering converts raw events into behavioral signals by comparing each event against the user's established baseline. The model has no access to the raw event stream — it only sees these derived features.

Feature	Type	Description
time_diff	Time	Seconds since user's last event
unusual_time	Time	1 if current hour deviates more than 6 hours from user's personal average active hour
trade_zscore	Trade	Z-score of current trade volume vs user's rolling mean and std (window = 5)

amount_zscore	Financial	Z-score of current deposit or withdrawal vs user's rolling baseline (window = 5)
session_event_count	Session	Number of events within the current 30-minute session window
recent_trade_count	Session	Number of trades in the last 5 events
ip_changed	IP	1 if IP address differs from previous event
ip_change_freq	IP	Rolling mean of ip_changed over last 5 events
pnl_volatility	Trading	Rolling std of trade_volume x margin over last 5 trades - measures erratic risk exposure
instrument_freq	Trading	Count of consecutive same-instrument trades in the last 5 trades. Detects single-instrument concentration.
withdraw_after_deposit	Financial	1 if current event is a withdrawal and a deposit occurred in the last 5 events
event_type_encoded	Categorical	Fixed integer encoding of event type using a deterministic map
instrument_encoded	Categorical	Fixed integer encoding of instrument using a deterministic map

Rolling statistics for trade volume and transaction amounts are computed on relevant rows only, then forward-filled per user. This ensures all events — including logins and KYC updates — carry the user's most recent behavioral baseline rather than a zero placeholder.

Categorical encoding uses fixed hardcoded maps rather than pandas cat.codes, ensuring consistent encoding between training and inference regardless of which categories appear in a given run.

7. Models

7.1 Isolation Forest

Isolation Forest isolates observations by randomly selecting a feature and a split value. Anomalous points require fewer splits to isolate and receive lower anomaly scores. It operates on individual events and requires no sequence context.

- 100 estimators, contamination = 0.03
- Input: 13 engineered features per event
- Output: decision_function score normalised to a 0 to 1 risk score
- Chosen because it handles high-dimensional tabular data without labeled training data and is fast enough for real-time inference

7.2 LSTM Autoencoder

The LSTM Autoencoder learns to reconstruct sequences of normal user behavior. At inference time, sequences that deviate from learned patterns produce high reconstruction errors, flagged as anomalies. This captures temporal dependencies that point-based models cannot detect.

- Architecture: LSTM encoder (64 units) -> RepeatVector -> LSTM decoder (64 units) -> Dense output
- Sequence length: 10 consecutive events per user
- Threshold: 97th percentile of training reconstruction errors, saved as lstm_threshold.pkl

- Requires at least 10 events before activating — new users are evaluated by Isolation Forest only until history builds

7.3 Final decision

Both models run independently. The final anomaly flag uses OR logic — if either model flags an event it is marked anomalous. If both flag it the alert level is elevated to high. The risk score is derived from the Isolation Forest decision function. The LSTM contributes a binary anomaly flag.

Property	Isolation Forest	LSTM Autoencoder
Anomaly type	Point anomalies	Sequence anomalies
Context window	Single event	10 consecutive events
Min history needed	None	10 events
Inference speed	Very fast	Fast after warmup
Output used	Risk score + anomaly flag	Anomaly flag only

8. Real-Time Processing

Training is batch-based on the full dataset. Streaming simulation applies to inference only — the API processes each event sequentially, maintaining per-user history in fakeredis and computing all features live against that history.

Per-event processing steps:

- Producer streams event into Kafka
- Consumer receives event and forwards it to POST /predict
- API retrieves stored history for this user_id from fakeredis
- Compute all 13 features using compute_features() against that history
- Scale features using the pre-fitted MinMaxScaler
- Run Isolation Forest on the single scaled feature vector
- If 10 or more events exist in feature history, run LSTM on the sequence
- Compute top contributing features from Isolation Forest
- Append current event to user history, capped at the last 10 events
- Return final_anomaly, risk_score, and reason

9. API Reference

GET /

Health check. Returns: {"message": "API is running"}

POST /predict

Accepts a single user event and returns anomaly detection results.

Request body:

```
{
  "user_id": 1,
  "timestamp": "2026-03-15T10:30:00",
  "event_type": "trade",
```

```
"trade_volume": 1200.0,
"amount": 0,
"margin": 500.0,
"instrument": "EURUSD",
"ip_address": "45.123.12.89"
}
```

Response:

```
{
  "final_anomaly": 1,
  "risk_score": 0.4731,
  "reason": "ip change + high trade spike"
}
```

For a meaningful prediction, send a sequence of normal events for the same user_id first to build a behavioral baseline, then send the suspicious event. A single isolated event will score low because compute_features() has no history to compare against.

10. Explainability

Every prediction includes a reason string derived from the top contributing features identified by the Isolation Forest perturbation method. For each feature, the model score is recomputed with that feature zeroed out — the features whose removal most reduces the anomaly score are ranked as the top contributors and mapped to plain-English labels.

Reason	Feature
high trade spike	trade_zscore
unusual transaction	amount_zscore
withdrawal after recent deposit	withdraw_after_deposit
ip change	ip_changed
frequent ip switching	ip_change_freq
activity at unusual time	unusual_time
trade burst	recent_trade_count
high session activity	session_event_count
high pnl volatility	pnl_volatility
instrument concentration	instrument_freq

11. Assumptions, Trade-offs, and Limitations

Assumptions

- User behavior is consistent enough within the simulation period that rolling statistics over 5 events capture a meaningful personal baseline
- The 3% anomaly rate approximates real-world fraud prevalence in a retail forex context
- trade_volume x margin is an adequate proxy for PnL exposure in the absence of live market price data
- A 30-minute gap between events is a reasonable session boundary

- Each user's active hour window remains stable over the simulation period — behavioral drift over time is not modeled

Trade-offs

- Isolation Forest operates on single events and cannot detect anomalies that only emerge across a sequence — this is why the LSTM is included alongside it
- LSTM requires at least 10 events before it activates — new users are evaluated by Isolation Forest only during the cold-start period
- The risk score is derived from Isolation Forest only. The LSTM contributes a binary anomaly flag
- Training is batch-based on the full dataset. True online learning — where the model updates incrementally with each new event — is not supported by Isolation Forest or LSTM Autoencoder in their standard implementations
- The LSTM threshold is fixed at the 97th percentile of training errors and does not adapt over time. As user behavior evolves, this threshold may need periodic recalibration

Limitations

- User history resets on every server restart since fakeredis is in-memory — swap for real Redis in production for persistence
- The API has no authentication — in a real deployment only authorised systems should be able to call it
- Alerts are returned as API responses only — they are not pushed anywhere. A real compliance team would receive them via a message queue or dashboard
- The model is trained once and never updated. If user behavior shifts over time the model becomes stale and will need retraining
- `instrument_freq` only catches consecutive same-instrument repetition. A user who alternates between two instruments would not be flagged even if they never diversify
- Coordinated activity across multiple accounts — such as mirror trades or shared IPs — is not detected since the system looks at each user independently
- PnL volatility is approximated using trade volume and margin since actual profit and loss requires live market price data

12. Path to Production

The current system works as a single-process prototype. The main gaps to address before production are:

- Replace fakeredis with real Redis so user history persists across restarts and scales across multiple servers
- Add API authentication so only authorised systems can call `/predict`
- Publish high-risk alerts to Kafka or RabbitMQ instead of only returning them in the API response
- Add monitoring to track prediction latency and anomaly rate over time, and trigger model retraining when drift is detected

13. Tech Stack

Layer	Tool	Purpose
Data	Pandas, NumPy	Dataset generation and feature engineering
ML - baseline	scikit-learn	Isolation Forest
ML - advanced	TensorFlow / Keras	LSTM Autoencoder
Scaling	scikit-learn	MinMaxScaler feature normalization
Serialization	joblib	Model, scaler, threshold persistence
API	FastAPI + Pydantic	REST endpoint with typed input validation
Server	Uvicorn	ASGI server
Streaming	Kafka + kafka-python	Event streaming pipeline
State	fakeredis -> Redis	Per-user history persistence